

AIDev: Studying AI Coding Agents on GitHub

Hao Li
Queen's University
Kingston, ON, Canada
hao.li@queensu.ca

Haoliang Zhang
Queen's University
Kingston, ON, Canada
haoxiang.zhang@queensu.ca

Ahmed E. Hassan
Queen's University
Kingston, ON, Canada
ahmed@cs.queensu.ca

Abstract

AI coding agents are rapidly transforming software engineering by performing tasks such as feature development, debugging, and testing. Despite their growing impact, the research community lacks a comprehensive dataset capturing how these agents are used in real-world projects. To address this gap, we introduce AIDev, a large-scale dataset focused on agent-authored pull requests (Agentic-PRs) in real-world GitHub repositories. AIDev aggregates 932,791 Agentic-PRs produced by five agents: OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code. These PRs span 116,211 repositories and involve 72,189 developers. In addition, AIDev includes a curated subset of 33,596 Agentic-PRs from 2,807 repositories with over 100 stars, providing further information such as comments, reviews, commits, and related issues. This dataset offers a foundation for future research on AI adoption, developer productivity, and human-AI collaboration in the new era of software engineering.

CCS Concepts

• Software and its engineering; • Information systems → Data mining;

Keywords

AI Agent, Agentic AI, Coding Agent, Agentic Coding, Software Engineering Agent

ACM Reference Format:

Hao Li, Haoliang Zhang, and Ahmed E. Hassan. 2026. AIDev: Studying AI Coding Agents on GitHub. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3797249>

1 High-Level Overview

The vision of AI Teammates [13] and recent evidence of their adoption in practice [19] signal a major transition in software engineering (SE). Coding Agents are increasingly acting as AI Teammates that participate in core development workflows. They now contribute thousands of pull requests (PRs)¹ daily, becoming routine actors in collaborative software development. This shift marks the emergence of SE 3.0 [13], where human-AI collaboration is deeply integrated into real-world projects.

Figure 1 illustrates how a Coding Agent operates within a real GitHub workflow. In this example, the agent (GITHUB COPILOT)

is assigned an issue, generates a code patch, and submits a PR with a detailed description. A human reviewer provides feedback, which the agent addresses in a follow-up commit and replies.² This interaction showcases the emerging dynamics of human-AI collaboration in software development, where Coding Agents not only contribute code but also remain engaged in the review process.

To support systematic study of this paradigm shift, we introduce AIDev, a large-scale dataset of Agentic-PRs from real-world GitHub projects. AIDev comprises 932,791 Agentic-PRs authored by five agents: OPENAI CODEX, DEVIN, GITHUB COPILOT, CURSOR, and CLAUDE CODE, across 116,211 repositories involving 72,189 developers (dataset cutoff: August 1, 2025). Each PR is linked to its corresponding repository and developer, along with additional metadata. For deeper analysis, we curated a subset of 33,596 Agentic-PRs from 2,807 repositories with more than 100 GitHub stars. This enriched subset provides review comments, commit-level diffs, event timelines, and related issues. AIDev enables research on adoption, quality, review, and risks of Coding Agents.

2 Internal Structure

Table 1 summarizes the major tables, their sizes, and the types of artifacts they contain. The AIDev dataset provides PR-level, repository-level, and developer-level metadata. The curated subset of repositories with more than 100 GitHub stars provides enriched artifacts such as inline review comments, commit-level diffs, linked issues, and full PR timelines. To support further analysis, the dataset also includes automated annotations of PR purpose (e.g., bug fix, feature, documentation), following the Conventional Commits categories. The complete relational schema is available in our dataset on Hugging Face and Zenodo (see Section 3).

3 How to Access (Links)

The AIDev dataset is available for download on Hugging Face and Zenodo. On Hugging Face, the dataset can be explored interactively through the “Data Studio” interface, which supports in-browser SQL queries. For reproducibility and ease of use, we also provide example Jupyter notebooks with ready-to-use Google Colab links in our GitHub repository. These notebooks demonstrate how to download, filter, and analyze the dataset. The related links are provided below:

- Hugging Face: <https://huggingface.co/datasets/hao-li/AIDev>
- Zenodo: <https://doi.org/10.5281/zenodo.16899501>
- GitHub: https://github.com/SAILResearch/AI_Teammates_in_SE3

¹We refer to PRs authored by Coding Agents as Agentic-PRs.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2474-9/26/04
<https://doi.org/10.1145/3793302.3797249>

²Not all Coding Agents currently support addressing review comments.

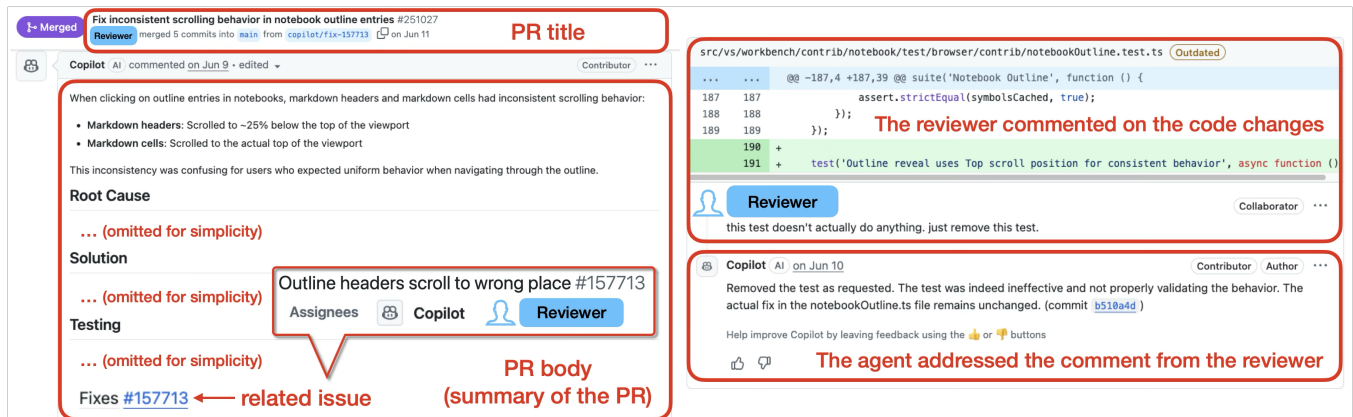


Figure 1: Example of an Agentic-PR on GitHub. The Coding Agent (GITHUB COPILOT) authored a pull request, received feedback from a human reviewer, and addressed the comment in a follow-up commit.

Table 1: Overview of the AIDEV Dataset

	Table	# Records	Content
Core Metadata	all_pull_request	932,791	Pull request metadata (title, body, agent, state, timestamps, repository, user)
	all_repository	116,211	Repository-level metadata (name, license, language, URL, stars, forks)
	all_user	72,189	Developer/user metadata (login, followers, creation date)
The following is a subset of PRs from repositories with more than 100 GitHub stars			
Core Metadata	pull_request	33,596	Same fields as all_pull_request, restricted to curated subset
	repository	2,807	Same fields as all_repository, restricted to curated subset
	user	1,796	Same fields as all_user, restricted to curated subset
Comments & Reviews	pr_comments	39,122	Discussion-style comments on PRs (author, body, timestamp)
	pr_reviews	28,875	Review verdicts (e.g., approve or request changes) with metadata (author, body, timestamp)
	pr_review_comments	19,450	Inline code review comments with file-level context (path, diff hunk, timestamp)
Commits & Diffs	pr_commits	88,576	Commits linked to PRs with metadata (SHA, author, message)
	pr_commit_details	711,923	File-level commit diffs including additions, deletions, and patches
Issues & Events	related_issue	4,923	Mapping between PRs and related issues
	issue	4,614	GitHub issues related to PRs (title, body, state, user, timestamps)
	pr_timeline	325,500	Full PR event history (e.g., committed, closed, merged, labeled, reviewed)
Annotation	pr_task_type	33,596	Automated classification of PR purpose (Conventional Commits categories, GPT-based)

4 Potential Research Questions

The AIDEV dataset opens avenues for a wide range of research opportunities into the role of Coding Agents in SE. We outline example research questions below.

4.1 Adoption and Practices

- (1) Who adopts Coding Agents on GitHub (e.g., newcomers vs. experienced developers)? How do adoption patterns vary across repositories and ecosystems?
- (2) What practices (e.g., PR size, task type, and commit granularity) correlate with the quality of Agentic-PRs? How can these practices inform concrete guidelines for developers to work with Agentic-PRs?

- (3) How can we identify developers who use Coding Agents most effectively? How can we build profiles for these developers? How can we understand their strategies and translate these insights into practical guidance to help others improve their skills and productivity?

4.2 Code Patch Characteristics

- (1) How do Agentic-PRs change code (e.g., additions, deletions, files touched)? How consistent are their descriptions with the actual code changes?
- (2) To what extent do Agentic-PRs introduce original code versus reusing existing snippets? What are the implications for maintainability?
- (3) How do code-change patterns in Agentic-PRs differ from Human-PRs (additions vs. deletions, refactorings, file types

touched, originality vs. reuse/copying), and what are the implications for software diversity and maintainability?

- (4) Do Agentic-PRs conform to project conventions and CI checks (linters, formatters, license headers)? How does conformance relate to review effort and merge outcomes?

4.3 Testing Behavior

- (1) How frequently do Coding Agents contribute tests? What types (e.g., unit, integration, end-to-end) are most common? What is the test-to-code churn ratio across ecosystems?
- (2) When tests are missing in initial Agentic-PRs, do developers intervene to ensure reliable software testing (via follow-up commits or related PRs)?

4.4 Review Dynamics

- (1) What aspects of Agentic-PRs (e.g., correctness, style, security, testing) receive the most attention during review?
- (2) To what extent do Coding Agents address review comments? Which comment types are challenging for agents to resolve?
- (3) How can understanding review gaps in AI-generated PRs guide humans in developing new skills for effective collaboration in coding, testing, and reviewing?

4.5 Failure Patterns and Risks

- (1) What common failure patterns and code quality issues appear in Agentic-PRs? Why do they occur? How can we leverage these insights to reduce failure rates, optimize human-AI collaboration, and improve AI model training that prioritizes learning from mistakes?
- (2) How well can early signals (e.g., PR description, touched paths, and patch characteristics) predict Agentic-PRs rejection or review effort?
- (3) How frequently do Agentic-PRs introduce or mitigate security vulnerabilities?
- (4) Which Agentic-PRs reach production and persist (i.e., are not reverted or hotfixed)? How do outcomes vary by language, repository maturity, and task type?
- (5) What security issues are more prevalent in agent-authored code (e.g., insecure APIs, dependency risks, secrets exposure), and how should review and security practices adapt?

5 Related Work

5.1 AI Coding Agents

Recent advances in large language models (LLMs) have enabled coding agents such as SWE-agent [44], OpenHands [39], and AutoCodeRover [46] that couple LLMs with planning, tool invocation, and repository-level reasoning, enabling tasks such as issue resolution, iterative repair, and test-driven development. Benchmark suites like SWE-bench [15] and its derivatives evaluate whether agents can resolve real GitHub issues end to end, strengthening the link between academic evaluation and production-relevant outcomes.

The technical architectures underlying agentic systems differ from traditional completion-based models. Core designs integrate environment awareness, long-horizon planning, and structured tool

use via carefully engineered agent-computer interfaces that expose file I/O, build and test execution, code search, and version-control operations [20, 44]. Multi-agent decompositions further segment responsibilities across planning, code editing, navigation, and execution, yielding higher success rates on open-source issue benchmarks than single-agent baselines [27]. Planning and refinement mechanisms, such as dynamic action resampling, pseudocode-style plan generation, and proposer-ranker loops, improve consistency and robustness across multi-step workflows [11, 37, 40].

Despite rapid progress, important limitations remain. Security analyses have shown elevated rates of vulnerabilities in AI-generated code, raising the stakes as agents obtain write or commit permissions in real repositories [18, 26, 30]. Usability studies identify challenges related to latency, context handling, and user trust, particularly when agents act proactively or at repository scale [8, 28]. Position papers argue that achieving programming with trust will require constraining autonomy through verifiable analyses, richer testing, and auditable traces of thought, action, and result trajectories [4]. Work on RepairAgent and related systems illustrates how structured finite-state workflows and tool gating can align agent behavior with developer expectations while improving reliability [3, 37].

Taken together, prior research depicts a field transitioning from token-level assistance toward agentic software engineering [12], in which LLM-driven systems plan, execute, and iterate over complex development tasks with varying degrees of autonomy [16, 38]. However, much of the existing evidence derives from controlled studies, benchmarks, or small-scale deployments. This gap motivates our AIDev dataset, which captures Agentic-PRs at scale across real GitHub repositories. By linking PR metadata with review artifacts, commit diffs, issue relations, and event timelines, AIDev provides a foundation for studying adoption, quality, review dynamics, and risks of Coding Agents in the wild.

5.2 Automated Contributions in Open Source Software

Prior work on automation in open source software has examined rule-based tools and development bots that react to well-defined triggers, operate within narrow scopes, and surface changes through templated interactions such as scripted comments or dependency update pull requests [10, 31]. These systems have helped scale maintenance work, most prominently in dependency management, but they rarely exhibit the kind of initiative, contextual reasoning, or dialogical interaction seen in human contributors [5, 10].

Bot-generated PRs tend to have lower acceptance rates, slower interactions, and well-documented issues with noise and notification fatigue, particularly in the context of dependency updates [14, 42]. These patterns have been linked to limited trust, brittle automation, and interruptions to established workflows [9, 41]. By contrast, recent observations of LLM use within PR workflows indicate that developers turn to these tools for larger, more complex changes. PRs associated with ChatGPT involvement are substantially slower to close and entail heavier review, suggesting qualitatively different kinds of contributions than routine maintenance [6, 7]. For dependency and repair bots, rejection often stems from volume, false alarms, or brittle patches [14, 29, 35]. Early evidence suggests that

LLM involvement correlates with heavier review workloads and longer time-to-merge, consistent with a shift from maintenance micro-changes to higher-stakes modifications [6, 7, 25].

5.3 Empirical Studies of AI Adoption and Developer Productivity

Field and observational studies complement surveys by revealing how developers actually interact with AI tools. In practice, many engineers rely on AI tools more for guidance, exploration, and sense-making than for drop-in code generation, weaving suggestions into existing validation and repair routines [17]. Interview-based investigations with early adopters broaden this picture across people, processes, and products, noting both workflow accelerations and socio-technical frictions as teams integrate LLM assistance into the software lifecycle [33]. Laboratory and think-aloud studies sharpen these insights: although participants often prefer working with tools like GitHub Copilot, measurable efficiency gains are inconsistent when tasks require deep comprehension or debugging of suggested code [36]. Grounded analyses of interaction further distinguish between acceleration, using AI tools to speed routine authoring, and exploration, using them to plan structure or discover APIs [1]. Recent work also foregrounds the affective dimension of AI tool use, documenting frustration and emotional strain that, while common, rarely lead to abandonment of these tools [23].

Telemetry and usage analytics offer population-scale evidence about real-world behavior. By linking IDE measurements with self-reports, Ziegler et al. [47] demonstrate that the acceptance rate of shown suggestions is a stronger predictor of perceived productivity than coarse output metrics, and that acceptance varies markedly across users and over time. Eye-tracking and IDE-instrumented experiments additionally show that awareness of code provenance changes behavior: when developers know a snippet was AI-generated, they search more, validate more, and experience higher cognitive load while nonetheless improving performance on some tasks [34]. Natural experiments on platform-wide rollouts provide convergent evidence for ecosystem-level impact: when Copilot selectively supported certain languages on GitHub, contribution volumes shifted in ways consistent with augmented collaborative innovation for supported languages [45]. Complementary analyses map AI use to specific tasks, including bug fixing and testing, suggesting that effectiveness depends on the surrounding validation and review practices into which suggestions are embedded [22, 32, 43].

Direct measurements of productivity remain mixed but cautiously positive, reflecting the inherent difficulty of quantifying developer work. Controlled studies frequently find improvements in perceived productivity and quality of starting points, even when completion time or task success rates do not uniformly improve [1, 36]. Broader evaluation frameworks such as RealHumanEval [24] seek to capture multidimensional outcomes across participants, tasks, and assistance modalities. Quasi-experimental evidence from policy shocks likewise indicates measurable gains: for example, a temporary regional restriction on LLM access enabled an estimate of a 6.4% productivity reduction during the ban, with heterogeneous effects by experience level [2]. Systematic reviews synthesize

these threads, noting that while most studies now adopt multidimensional perspectives, the field still leans heavily on self-reports and short-term, individual-level designs [21].

References

- [1] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1 (2023), 85–111. doi:10.1145/3586030
- [2] Sardar Bonabi, Sarah Bana, Vijay Gurbaxani, and Tingting Nian. 2025. Beyond Code: The Multidimensional Impacts of Large Language Models in Software Development. *CoRR* abs/2506.22704 (2025). arXiv:2506.22704 doi:10.48550/ARXIV.2506.22704
- [3] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. (2025), 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [4] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. *CoRR* abs/2506.18824 (2025). arXiv:2506.18824 doi:10.48550/ARXIV.2506.18824
- [5] Nathan Cassee, Bogdan Vasilescu, and Alexander Serebrenik. 2020. The Silent Helper: The Impact of Continuous Integration on Code Reviews. In *27th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2020, London, ON, Canada, February 18–21, 2020*, Kostas Kontogiannis, Foutse Khomh, Alexander Chatzigeorgiou, Marios-Eleftherios Fokaefs, and Minghui Zhou (Eds.). IEEE, 423–434. doi:10.1109/SANER48275.2020.9054818
- [6] Arifa I. Champa, Md. Fazle Rabbi, Costain Nachuma, and Minhaz F. Zibran. 2024. ChatGPT in Action: Analyzing Its Use in Software Development. In *21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15–16, 2024*, Diomidis Spinellis, Alberto Bacchelli, and Eleni Constantinou (Eds.). ACM, 182–186. doi:10.1145/3643991.3645077
- [7] Moataz Chouchen, Narjes Bessghaier, Mahi Begoug, Ali Ouni, Eman Alomar, and Mohamed Wiem Mkaouer. 2024. How Do Software Developers Use ChatGPT? An Exploratory Study on GitHub Pull Requests. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 212–216. doi:10.1145/3643991.3645084
- [8] Philipp Eibl, Sadra Sabouri, and Souti Chattopadhyay. 2025. Exploring the Challenges and Opportunities of AI-assisted Codebase Generation. *CoRR* abs/2508.07966 (2025). arXiv:2508.07966 doi:10.48550/ARXIV.2508.07966
- [9] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. 2020. An empirical study of bots in software development: characteristics and challenges from a practitioner's perspective. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8–13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 445–455. doi:10.1145/3368089.3409680
- [10] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. 2022. Dependency management bots in open-source systems - prevalence and adoption. *PeerJ Comput. Sci.* 8 (2022), e849. doi:10.7717/PEERJ-CS.849
- [11] Muhammad Haseeb. 2025. Context Engineering for Multi-Agent LLM Code Assistants Using Elicit, NotebookLM, ChatGPT, and Claude Code. *CoRR* abs/2508.08322 (2025). arXiv:2508.08322 doi:10.48550/ARXIV.2508.08322
- [12] Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. 2025. Agentic Software Engineering: Foundational Pillars and a Research Roadmap. arXiv:2509.06216 [cs.SE]
- [13] Ahmed E. Hassan, Gustavo A. Oliva, Dayi Lin, Boyuan Chen, and Zhen Ming Jiang. 2024. Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap. arXiv:2410.06107 [cs.SE]
- [14] Runzhi He, Hao He, Yuxia Zhang, and Minghui Zhou. 2023. Automating Dependency Updates in Practice: An Exploratory Study on GitHub Dependabot. *IEEE Trans. Software Eng.* 49, 8 (2023), 4004–4022. doi:10.1109/TSE.2023.3278129
- [15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. https://openreview.net/forum?id=VTF8yNQm66
- [16] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. *CoRR* abs/2408.02479 (2024). arXiv:2408.02479 doi:10.48550/ARXIV.2408.02479
- [17] Ranim Khojah, Mazen Mohamad, Philipp Leitner, and Francisco Gomes de Oliveira Neto. 2024. Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1819–1840. doi:10.1145/3660788
- [18] Matous Kozák, Roshanak Zilouchian Moghaddam, and Siva Sivaraman. 2025. When Developer Aid Becomes Security Debt: A Systematic Analysis of Insecure Behaviors in LLM Coding Agents. *CoRR* abs/2507.09329 (2025). arXiv:2507.09329

- doi:10.48550/ARXIV.2507.09329
- [19] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Team-mates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv:2507.15003 [cs.SE]*
- [20] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion 2025, Clarion Hotel Trondheim, Trondheim, Norway, June 23–28, 2025*, Leonardo Montecchi, Jingyue Li, Denys Poshyvanyk, and Dongmei Zhang (Eds.). ACM, 238–249. doi:10.1145/3696630.3728549
- [21] Amr Mohamed, Maram Assi, and Mariam Guizani. 2025. The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review. *CoRR abs/2507.03156 (2025)*. *arXiv:2507.03156* doi:10.48550/ARXIV.2507.03156
- [22] Mauricio Monteiro, Bruno Castelo Branco, Samuel Silvestre, Guilherme Avelino, and Marco Túlio Valente. 2025. NoCodeGPT: A No-Code Interface for Building Web Apps With Language Models. *Softw. Pract. Exp.* 55, 8 (2025), 1408–1424. doi:10.1002/SPE.3432
- [23] Cristina Martinez Montes and Ranim Khojah. 2025. Emotional Strain and Frustration in LLM Interactions in Software Engineering. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering, EASE 2025, Istanbul, Turkey, June 17–20, 2025*, Muhammad Ali Babar, Ayse Tosun, Stefan Wagner, and Viktoria Stray (Eds.). ACM, 193–204. doi:10.1145/3756681.3756951
- [24] Hussein Moazzam, Valerie Chen, Mohammed Alsobay, Subhro Das, Sebastian Zhao, Dennis Wei, Manish Nagireddy, Prasanna Sattigeri, Ameet Talwalkar, and David A. Sontag. 2025. The RealHumanEval: Evaluating Large Language Models' Abilities to Support Programmers. *Trans. Mach. Learn. Res.* 2025 (2025). <https://openreview.net/forum?id=hGaWq5Buj7>
- [25] Daniel Ogenwot and John Businge. 2025. PatchTrack: A Comprehensive Analysis of ChatGPT's Influence on Pull Request Outcomes. *CoRR abs/2505.07700 (2025)*. *arXiv:2505.07700* doi:10.48550/ARXIV.2505.07700
- [26] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Commun. ACM* 68, 2 (2025), 96–105. doi:10.1145/3610721
- [27] Huy Nhat Phan, Phong X. Nguyen, and Nghi D. Q. Bui. 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. *CoRR abs/2409.16299 (2024)*. *arXiv:2409.16299* doi:10.48550/ARXIV.2409.16299
- [28] Kevin Pu, Daniel Lazaro, Ian Arawjo, Haijun Xia, Ziang Xiao, Tovi Grossman, and Yan Chen. 2025. Assistance or Disruption? Exploring and Evaluating the Design and Trade-offs of Proactive AI Programming Support. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI 2025, Yokohama Japan, 26 April 2025– 1 May 2025*, Naomi Yamashita, Vanessa Evers, Koji Yatani, Sharon Xianghua Ding, Bongshin Lee, Marshini Chetty, and Phoebe O. Toups Dugas (Eds.). ACM, 152:1–152:21. doi:10.1145/3706598.3713357
- [29] Benjamin Rombaut, Filipe Roseiro Cogo, Bram Adams, and Ahmed E. Hassan. 2023. There's no Such Thing as a Free Lunch: Lessons Learned from Exploring the Overhead Introduced by the Greenkeeper Dependency Bot in Npm. *ACM Trans. Softw. Eng. Methodol.* 32, 1 (2023), 11:1–11:40. doi:10.1145/3522587
- [30] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. Are AI-Generated Fixes Secure? Analyzing LLM and Agent Patches on SWE-bench. *CoRR abs/2507.02976 (2025)*. *arXiv:2507.02976* doi:10.48550/ARXIV.2507.02976
- [31] Emad Shihab, Stefan Wagner, Marco Aurélio Gerosa, Mairieli Santos Wessel, and Jordi Cabot. 2022. The Present and Future of Bots in Software Engineering. *IEEE Softw.* 39, 5 (2022), 28–31. doi:10.1109/MS.2022.3176864
- [32] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An Analysis of the Automatic Bug Fixing Performance of ChatGPT. In *IEEE/ACM International Workshop on Automated Program Repair, APR@ICSE 2023, Melbourne, Australia, May 16, 2023*. IEEE, 23–30. doi:10.1109/APR59189.2023.00012
- [33] Benyamin T. Tabarsi, Heidi Reichert, Ally Limke, Sandeep Kuttal, and Tiffany Barnes. 2025. LLMs' Reshaping of People, Processes, Products, and Society in Software Development: A Comprehensive Exploration with Early Adopters. *CoRR abs/2503.05012 (2025)*. *arXiv:2503.05012* doi:10.48550/ARXIV.2503.05012
- [34] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. A Study on Developer Behaviors for Validating and Repairing LLM-Generated Code Using Eye Tracking and IDE Actions. *CoRR abs/2405.16081 (2024)*. *arXiv:2405.16081* doi:10.48550/ARXIV.2405.16081
- [35] Simon Uri, Zhongxing Yu, Lionel Seinturier, and Martin Monperrus. 2018. How to design a program repair bot?: insights from the repairator project. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Frances Paulisch and Jan Bosch (Eds.). ACM, 95–104. doi:10.1145/3183519.3183540
- [36] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI '22: CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA, 29 April 2022 - 5 May 2022, Extended Abstracts*, Simone D. J. Barbosa, Cliff Lampe, Caroline Appert, and David A. Shamma (Eds.). ACM, 332:1–332:7. doi:10.1145/3491101.3519665
- [37] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram K. Rajamani. 2024. CORE: Resolving Code Quality Issues using LLMs. *Proc. ACM Softw. Eng.* 1, FSE (2024), 789–811. doi:10.1145/3643762
- [38] Huaning Wang, Jingzhi Gong, Huawei Zhang, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. *CoRR abs/2508.11126 (2025)*. *arXiv:2508.11126* doi:10.48550/ARXIV.2508.11126
- [39] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=OJd3ayDDoF>
- [40] Jiaxin Wen, Jian Guan, Hongning Wang, Wei Wu, and Minlie Huang. 2024. CodePlan: Unlocking Reasoning Potential in Large Language Models by Scaling Code-form Planning. *CoRR abs/2409.12452 (2024)*. *arXiv:2409.12452* doi:10.48550/ARXIV.2409.12452
- [41] Mairieli Santos Wessel, Igor Wiese, Igor Steinmacher, and Marco Aurélio Gerosa. 2021. Don't Disturb Me: Challenges of Interacting with Software Bots on Open Source Software Projects. *Proc. ACM Hum. Comput. Interact.* 5, CSCW2 (2021), 301:1–301:21. doi:10.1145/3476042
- [42] Marvin Wyrich, Raoul Ghit, Tobias Haller, and Christian Müller. 2021. Bots Don't Mind Waiting, Do They? Comparing the Interaction With Automatically and Manually Created Pull Requests. In *3rd IEEE/ACM International Workshop on Bots in Software Engineering, BotSE@ICSE 2021, Madrid, Spain, June 4, 2021*. IEEE, 6–10. doi:10.1109/BOTSE52550.2021.00009
- [43] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023*. IEEE, 1482–1494. doi:10.1109/ICSE48619.2023.00129
- [44] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- [45] Doron Yeverechyahu, Raveesh Mayya, and Gal Oestreicher-Singer. 2024. The Impact of Large Language Models on Open-source Innovation: Evidence from GitHub Copilot. In *Proceedings of the 45th International Conference on Information Systems, ICIS 2024, Advances in Methods, Theories, and Philosophy, Bangkok, Thailand, December 15–18, 2024*, Douglas R. Vogel, Heiko Gewald, Assadaporn Sapsomboon, Christy M. K. Cheung, Sven Laumer, and Jason Thatcher (Eds.). Association for Information Systems. <https://aisel.aisnet.org/icis2024/diginnoventren/diginnoventren/38>
- [46] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16–20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1592–1604. doi:10.1145/3650212.3680384
- [47] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *MAPS@PLDI 2022: 6th ACM SIGPLAN International Symposium on Machine Programming, San Diego, CA, USA, 13 June 2022*, Swarat Chaudhuri and Charles Sutton (Eds.). ACM, 21–29. doi:10.1145/3520312.3534864